

Gambit Closure 4.0.1

CUDA repair and final verification

Verified on the requested NVIDIA GPU. The revised CUDA executable compiles, matches the CPU reference throughout the disclosed runs, and passes all four requested Compute Sanitizer modes. Both fresh formal and full CPU runs pass.

Verification scope	Recorded result
Hardware coordinator	GPU_CPU_EXACT_PASS
Separate CPU coordinator	CPU_PASS_GPU_NOT_RUN
Native / Python tests	338,184 native checks; 28 Python methods (24 numerical + 4 coordinator)
Frozen trajectories	6 complete profiles; exact consensus-file hashes
CUDA equality and LUT readback	13 checked coordinator executions
SMT obligations	24 expected results; solver status PASS
Compute Sanitizer	PASS - memcheck, racecheck, initcheck, synccheck
Nsight Compute	Texture and global reports captured for review
Independent completion audit	PASS

What this establishes

The repaired implementation meets the executed conformance and instrumentation checks described in this report. SMT results concern the submitted algebraic targets; they are separate evidence from CPU and GPU execution.

It is not a machine-checked proof of the complete implementation, compiler, runtime or physical GPU. The solver logs have no independent second proof checker. The numerical results make no claim about subjective consciousness or physical perpetual operation.

Revision root

C:\RUUDDENDAASKOANAL\Gambit_Closure_4.0.1

All evidence paths in this report are relative to that directory. The original 4.0 release remains separately preserved.

Repairs and actual execution environment

Confirmed compilation defect

The original CU(call) macro declared a local variable named e. In CUDA calls containing the Engine argument e, that declaration shadowed the argument inside its own initializer. NVCC rejected the frozen source. The revised checker receives the completed cudaError_t result in an inline function; the macro now forwards the expression without introducing a local variable.

Diagnostic accuracy

The CUDA comparison now tracks the update index within the segment. A mismatch after the first update is attributed to that update, rather than always reporting zero. The evidence suite checks the failure path as well as matching executions.

Reproducible Windows verification

The coordinator accepts an explicit CUDA toolset for CMake, resolves supported NVIDIA forwarding launchers to their executable, and records per-command and per-stage status. This makes toolkit selection and instrumentation failures visible in the saved JSON and logs. CMake aligns host and CUDA targets on the static MSVC runtime. Consult the revision diff for the exact changes.

Component	Observed identity
GPU	NVIDIA GeForce RTX 5070 Ti Laptop GPU
Compute capability	12.0
Total VRAM reported by CUDA	12,820,480,000 bytes
NVIDIA driver	591.59
CUDA compiler	Toolkit 12.8; nvcc 12.8.61
Host C++ compiler	MSVC 19.44.35221.0
Python / platform	3.13.11 / Windows-11-10.0.26200-SP0
CMake CUDA toolset	cuda=12.8
Formal solver	Z3 version 4.15.4 - 64 bit

Source records: REVISION.json; revision diff; validation/full/logs/cuda_build.txt; validation/full/logs/cuda_configure.txt; validation/full/logs/device.txt; validation/full/logs/nvidia_smi.txt; validation/full/logs/nvcc_version.txt.

Exact CPU and GPU conformance

The fresh full coordinator ran 338,184 native checks and 28 Python methods: 24 numerical reference methods and 4 coordinator regressions. Each frozen profile below matched all of its expected consensus-file digests. The independent Python model and Boolean ALU were also compared on the four smaller profiles.

Frozen profile	Updates	Cells	Files	Compared realizations
micro_xx	64	64	5	native + CUDA + Python + gates
micro_xy	64	64	5	native + CUDA + Python + gates
verify_xx	64	2,112	5	native + CUDA + Python + gates
verify_xy	64	2,112	5	native + CUDA + Python + gates
laptop_xx	256	131,072	5	native + CUDA
laptop_xy	256	131,072	5	native + CUDA

Consensus files

initial.gbc, final.gbc, words.bin, counts.bin and summary.csv. Equality is byte for byte, with the expected hashes read from the original frozen manifest. run.json identifies execution scope and is outside this five-file equality target.

CUDA execution matrix

verify_xy variants: texture / block 32; texture / block 128; texture / block 256; texture / block 512; global / block 256.

Every checked CUDA execution reads back the full LUT and compares each update against the CPU implementation. The coordinator also runs a fresh repeated process, forces the PTX JIT path, restores a CPU-generated checkpoint into CUDA, and captures PTX plus SASS from the built executable.

Additional check	Evidence
17 + 47 checkpoint continuation	PASS
Invalid command-line cases	4 rejected
Fresh-process repeat	cuda_repeat.txt - exit 0
Forced PTX JIT	cuda_forced_ptx.txt - exit 0
CUDA capsule restart	cuda_resume.txt - exit 0

Results cover the disclosed profiles and probes. They do not exhaust every admitted state. Trace auditing checks emitted pulse/event/parity identities and endpoint count certificates; it does not reconstruct every past activity input.

Fresh formal solver results

Z3 version 4.15.4 - 64 bit: 22 expected UNSAT results and 2 expected SAT counterexamples. Each target returned its expected result with no solver error.

Submitted SMT target	Expected	Observed
full_adder	UNSAT	UNSAT
codec_local_and_bounds	UNSAT	UNSAT
memory_bounds	UNSAT	UNSAT
four_edge_outflow	UNSAT	UNSAT
edge_conservation	UNSAT	UNSAT
reaction_nonnegative_mass	UNSAT	UNSAT
phase_closure_32	UNSAT	UNSAT
phase_closure_512	UNSAT	UNSAT
phase_closure_2048	UNSAT	UNSAT
cooldown_quotient	UNSAT	UNSAT
hysteresis_total	UNSAT	UNSAT
latch_event_telescope	UNSAT	UNSAT
prefix_certificate_compose	UNSAT	UNSAT
ripple_add_8	UNSAT	UNSAT
ripple_add_17	UNSAT	UNSAT
ripple_add_32	UNSAT	UNSAT
ripple_add_64	UNSAT	UNSAT
ripple_less_8	UNSAT	UNSAT
ripple_less_32	UNSAT	UNSAT
restoring_divide_8	UNSAT	UNSAT
parity_three_word_homomorphism	UNSAT	UNSAT
hadamard_roundtrip	UNSAT	UNSAT
one_bit_not_sufficient	SAT	SAT
even_error_parity_counterexample	SAT	SAT

The SAT targets are intentional: the last emitted bit does not uniquely determine the next state, and two data-bit flips can preserve parity. These are constructive counterexamples to stronger claims, rather than failing obligations.

The formulas and expectations were not altered. Obligations establish their stated algebraic propositions in Z3 semantics; they do not model the whole C++/CUDA program. For example, the restoring-divider target checks eight-bit arithmetic.

Source: `formal/obligations.json`. Results and solver proof/model text: `validation/full/formal/report.json` and the adjacent target logs. The independently repeated initial run is under `validation/formal_initial/`.

Instrumentation and additional probes

Compute Sanitizer on the actual GPU

Mode	Result	Exact log summary
memcheck	PASS	ERROR SUMMARY: 0 errors
racecheck	PASS	RACECHECK SUMMARY: 0 hazards displayed (0 errors, 0 warnings)
initcheck	PASS	ERROR SUMMARY: 0 errors
synccheck	PASS	ERROR SUMMARY: 0 errors

The coordinator invoked each mode with --error-exitcode 1 on an eight-update verify_xy run. These checks are evidence for those instrumented executions; their finite coverage remains distinct from a proof of every possible run.

Additional boundary and failure-path evidence

Edge-probe status: PASS_REQUESTED_SCOPE. Source report: validation/gpu_edges/verification.json.

Probe group	Observed evidence
Additional exact comparisons	4: block 64 and feedback-off native/Python/texture/global comparisons
Invalid CUDA arguments	6 rejected without creating output directories
CUDA error handling	Wrapper contract passed; process-local hidden device reported its CUDA error
Injected cell-state mismatch	FAIL: CPU/GPU state mismatch at step 3, cell 0
Injected clock mismatch	FAIL: GPU/native closed-transition mismatch at step 3

Faults were inserted only into isolated diagnostic copies. Both stopped after three successful updates and produced no final capsule or completed-run record. The production source and expected outputs remained unchanged.

Nsight Compute captures

Fetch path	Captured artifact	Bytes
texture	validation/full/ncu_texture.ncu-rep	930,621
global	validation/full/ncu_global.ncu-rep	931,261

Both captures are available for inspection. No texture-cache speedup or throughput advantage is inferred. This implementation copies state to the host and compares it per update, so the verification run is not a throughput benchmark.

Logs: validation/full/logs/sanitize_*.txt and profile_*.txt. The .ncu-rep files preserve the collected device metrics.

Reproduction and evidence identity

Run from the revision root, with the installed toolchain on PATH. Choose unused output directories on every run. The supplied Z3 directory is local to this revision and was unpacked from an existing cached wheel after approval.

```
$env:PATH=(Join-Path (Get-Location) 'local_tools/z3-4.15.4.0/bin')+';'+$env:PATH
python tools/verify.py --cpu-only --formal --out validation/reproduce_cpu
python tools/verify.py --hardware --formal --sanitizers --profile --cmake-cuda-toolset cuda=12.8 --out
validation/reproduce_full
```

Additional regressions use tools/verify_gpu_edges.py with --cpu, --gpu, --contract, --out and --diagnostics; fault builds also require --core-lib and --host-compiler (see the script's --help). All 18 exact command argument arrays, including compiler and executable paths, are recorded in validation/gpu_edges/verification.json.

Preserved release and fixed expectations

All 196 files in the original frozen 4.0 manifest still match. The revision manifest independently matches 203 entries. The expected output manifest is identical in both directories.

After the main runs, only README wording and revision patch/metadata were corrected. The audit confirms all compiled sources, tests, verification tools and inputs match the tested snapshot: validation/full/verified_source_manifest.txt.

```
assets/closure_goldens.json SHA-256
1c3e4e7fc27f0f5d0a9c2760fb5b1fe3063dcbf3329e3453e4743d1c8713e4c3
```

Exact source and executable identities

```
src/cuda.cu
78525113c0a0b728690b9b13ed9864b04624e68fb2643409cc46e40415720e98
tools/verify.py
55bdabf5133fa0872571dd2f9b526323574f1f2f515534bf8ff4e7864ef13aae
SHA256SUMS.txt
5d6a1d2f1e17f6ad0c63f2768d07689ed10c6a3aa905e292cdf4e839b14b7e3b
validation/full/cuda_build/Release/gambit_cuda.exe
192627c30b47a0f0b2f60b2454b75c36a55846e7452422e1bcefdb1718843cc9
validation/full/build/Release/gambit.exe
a9c7427d0583402815213249b8d2eb58a911467914bd57a0ad1fe481060ff9c8
```

Evidence entry points

Record	Purpose
REVISION.json and REVISION.patch	Disclosed source changes and base identity
validation/full/verification.json	Hardware run, stages, commands and profile results
validation/cpu/verification.json	Separate full CPU and formal execution
validation/gpu_edges/verification.json	Boundary and failure-path probes
validation/completion_audit.json	Independent completion audit
local_tools/z3-4.15.4.0/PROVENANCE.json	Offline solver archive and extracted-file hashes

Hashes bind the identified bytes to this record. They do not establish authorship, authenticity, or reachable state history. Keep the logs and source beside the PDF when reproducing or reviewing this result.